

MLE-STAR Web Search Integration

Detailed implementation plan for DuckDuckGo search

Branch: feature/reconstruct-mle-star-openai

Generated: 2026-06-17

1. What is this document?

This document describes the exact changes needed to integrate DuckDuckGo web search into the MLE-STAR OpenAI-compatible reconstruction. It is written for someone who has not worked on the project before. It explains what each file does, why it needs to change, and what the new code should look like.

2. Project context

2.1 What is MLE-STAR?

MLE-STAR is an autonomous multi-agent system for machine-learning engineering. You give it a dataset and a task description, and a team of AI agents designs, trains, debugs, and improves a machine-learning solution for you. Each agent is powered by a large language model (LLM).

2.2 What is the reconstruction branch?

The original MLE-STAR was built on Google's ADK and Gemini. Our current environment cannot use those services, so the system was rebuilt to run on the university's OpenAI-compatible API. The reconstruction branch already implements the full pipeline: model retrieval, evaluation, merging, refinement, ensemble, data checking, and submission.

2.3 Where does web search fit?

Web search is used in the very first agent: the model retriever. Instead of asking the LLM 'what models do you know for this task?', the system first searches the web for recent effective approaches, papers, or Kaggle solutions. Those search results are then included in the prompt to the LLM, so the proposed models are grounded in real-world information.

3. Decisions for this integration

- Search provider: DuckDuckGo (no API key, no registration, free).
- Library: ddgs (formerly duckduckgo-search, Python package).
- Activation: opt-in. `use_web_search` defaults to False to avoid unexpected rate limits.
- Number of results sent to the LLM: 5.
- Provider is fixed as DuckDuckGo; no abstraction layer for other providers.
- The LLM remains OpenAI-compatible; web search is a separate utility.

4. High-level flow

After the integration, the model retrieval step will work like this:

1. The user sets `use_web_search=True` in config or via environment variable.
2. The pipeline starts and reaches `model_retriever_agent_1`.
3. The agent builds a search query from the task summary.
4. It calls `web_search_util.search_web(query, num_results=5)`.
5. The utility uses DuckDuckGo and returns a list of results.
6. The results are stored in `AgentState`.
7. The prompt `MODEL_RETRIEVAL_INSTR` now includes the search results.
8. The LLM proposes model candidates based on those results.
9. The rest of the pipeline continues unchanged.

5. Files to change

The integration touches the following files:

- `pyproject.toml` — add `duckduckgo-search` dependency.
- `machine_learning_engineering/shared_libraries/web_search_util.py` — new utility module.
- `machine_learning_engineering/shared_libraries/config.py` — add web search settings.
- `.env.example` — document new optional settings.
- `machine_learning_engineering/sub_agents/initialization/agent.py` — call web search before model retrieval.
- `machine_learning_engineering/sub_agents/initialization/prompt.py` — include search results in prompt.

- machine_learning_engineering/scripts/validate_web_search.py — keep as standalone validator.
- tests/ — add unit and integration tests.

6. Step-by-step changes

6.1 Install the dependency

First, install the DuckDuckGo search library in the existing virtual environment. The package is now distributed as ddgs on PyPI, although older documentation may still refer to duckduckgo-search.

```
.venv/bin/pip install ddgs
```

After installation, the validation script

machine_learning_engineering/scripts/validate_web_search.py should work if it imports from ddgs.

6.2 Add dependency to pyproject.toml

So that future installations include the library automatically, add it to the project dependencies.

```
dependencies = [
    "openai>=1.14.0",
    "python-dotenv>=1.0.1",
    "pydantic>=2.10.6",
    "numpy>=2.2.3",
    "pandas>=2.3.1",
    "scikit-learn>=1.7.1",
    "skrub",
    "torch>=2.7.1",
    "ddgs",
]
```

6.3 Create web_search_util.py

Create a new file that encapsulates all DuckDuckGo search logic. This keeps the rest of the codebase clean and makes it easy to find the search functionality.

```
"""Web search utility for MLE-STAR using DuckDuckGo."""

from typing import Any

from ddgs import DDGS

def search_web(query: str, num_results: int = 5) -> list[dict[str, Any]]:
    """Search DuckDuckGo and return a list of result dicts."""
    try:
        with DDGS() as ddgs:
            results = list(ddgs.text(query, max_results=num_results))
            return results
    except Exception:
        return []

def format_results_for_prompt(results: list[dict[str, Any]]) -> str:
    """Convert search results into a string suitable for an LLM prompt."""
    if not results:
        return "No web search results available."
    lines = []
    for i, item in enumerate(results, start=1):
        title = item.get("title", "No title")
        body = item.get("body", "").replace("\n", " ")
        lines.append(f"{i}. {title} - {body}")
    return "\n".join(lines)
```

Why this design? It separates the raw search call from prompt formatting. If we ever need to change how results are presented to the LLM, we only change format_results_for_prompt.

6.4 Update config.py

Add web search settings to the DefaultConfig dataclass. The default is False so users must explicitly opt in.

```
use_web_search: bool = False
web_search_num_results: int = 5
```

These values will be copied into AgentState during prepare_task, just like all other config values.

6.5 Update .env.example

Document the new optional setting so users know it exists.

```
# Optional: enable DuckDuckGo web search during model retrieval
USE_WEB_SEARCH=false
WEB_SEARCH_NUM_RESULTS=5
```

Note: because the value is read through config.py, USE_WEB_SEARCH must be parsed as a boolean. If you use os.environ.get directly, remember to convert 'true'/'false' strings to bool.

6.6 Modify initialization/agent.py

Two changes are needed in the initialization agent: import the utility and call it before model retrieval.

6.6.1 Add import

```
from machine_learning_engineering.shared_libraries import (
    code_util,
    common_util,
    config,
    debug_util,
    web_search_util,
)
```

6.6.2 Add helper function

```
def run_web_search_for_task(state, task_id) -> None:
    """Search the web for recent models/approaches for this task."""
    if not state.get("use_web_search", False):
        return
    task_summary = state.get("task_summary", "")
    query = f"effective machine learning models for {task_summary}"
    num_results = state.get("web_search_num_results", 5)
    results = web_search_util.search_web(query, num_results=num_results)
    state[f"web_search_results_{task_id}"] = results
    state[f"web_search_query_{task_id}"] = query
```

6.6.3 Update run_initialization_pipeline

```
# Summarize task
run_agent(..., "task_summarization_agent", ...)

# Optional web search before model retrieval
run_web_search_for_task(state, task_id)

# Retrieve/propose models
run_agent(..., f"model_retriever_agent_{task_id}", ...)
```

Why before model retrieval? Because the model retriever prompt needs the search results when it is built. The search runs only once per task, so the cost and latency are minimal.

6.7 Update initialization/prompt.py

The model retrieval prompt must include a placeholder for search results and instructions on how to use them.

```

MODEL_RETRIEVAL_INSTR = """# Competition
{task_summary}

# Web search results
{web_search_results}

# Your task
- Propose {num_model_candidates} simple and effective machine learning models for the above competition.
- Use the web search results above to inform your proposals.
- For California housing prices (a tabular regression task), prefer scikit-learn models...

# Requirement
- Provide the model name and a short, self-contained Python code snippet...

Use this exact JSON format (and nothing else):
[
  {"model_name": "Name of the model", "example_code": "from sklearn...\n..."}
]
"""

```

6.8 Update get_model_retriever_agent_instruction

This function builds the prompt. It now needs to read the search results from state and format them.

```

def get_model_retriever_agent_instruction(state, agent_name: str) -> str:
    task_summary = state.get("task_summary", "")
    num_model_candidates = state.get("num_model_candidates", 1)
    task_id = agent_name.split("_")[-1]
    results = state.get(f"web_search_results_{task_id}", [])
    web_search_results = web_search_util.format_results_for_prompt(results)
    return prompt.MODEL_RETRIEVAL_INSTR.format(
        task_summary=task_summary,
        num_model_candidates=num_model_candidates,
        web_search_results=web_search_results,
    )

```

When web search is disabled, results is an empty list and format_results_for_prompt returns a neutral message. The prompt still works.

7. Error handling and fallback

DuckDuckGo can fail or block requests. The integration must be resilient:

- search_web catches exceptions and returns an empty list, so the LLM still receives a prompt.
- If use_web_search is False, no search is performed and no API call is made.
- If the search returns empty results, the prompt includes 'No web search results available' and the LLM falls back to its own knowledge.
- The rest of the pipeline is unchanged, so a search failure never prevents submission.csv from being generated.

8. Tests to add

- Unit test for web_search_util.search_web using a mocked DDGS.text().
- Unit test for web_search_util.format_results_for_prompt with empty and non-empty results.
- Integration test that verifies web_search_results_1 is populated in state when use_web_search=True.
- Integration test that verifies no search is performed when use_web_search=False.
- Integration test that verifies the pipeline still completes if search_web returns empty.

9. Validation steps

After implementing the changes, validate in this order:

- Run the standalone validation script: `.venv/bin/python3 machine_learning_engineering/scripts/validate_web_search.py`
- Run unit tests: `.venv/bin/python3 -m pytest tests/`
- Run the full pipeline with `use_web_search=False`: behavior should be identical to before.
- Run the full pipeline with `use_web_search=True`: verify `web_search_results_1` appears in

final_state.json and submission.csv is generated.

10. Jira cards

MLE-WEB-101: Install and declare ddgs dependency

- Install ddgs in .venv.
- Add ddgs to pyproject.toml dependencies.
- Verify import works from project code.

MLE-WEB-102: Create web_search_util.py

- Implement search_web(query, num_results=5) using DDGS.
- Implement format_results_for_prompt(results).
- Add exception handling that returns empty results on failure.

MLE-WEB-103: Add web search configuration

- Add use_web_search and web_search_num_results to DefaultConfig.
- Update .env.example with optional settings.
- Ensure config values are copied into AgentState.

MLE-WEB-104: Integrate web search into model retriever

- Import web_search_util in initialization/agent.py.
- Add run_web_search_for_task(state, task_id) helper.
- Call helper before model_retriever_agent in run_initialization_pipeline.
- Update get_model_retriever_agent_instruction to read and format search results.

MLE-WEB-105: Update MODEL_RETRIEVAL_INSTR prompt

- Add {web_search_results} placeholder.
- Add instruction telling the LLM to use the search results.
- Ensure prompt renders correctly when search results are empty.

MLE-WEB-106: Add tests and validate end-to-end

- Add unit tests for web_search_util.
- Add integration tests for use_web_search True/False and empty results fallback.
- Run full pipeline with use_web_search=True on California Housing.
- Document score and token comparison against a run without web search.

11. Example usage

To enable web search, a user only needs to change one value in config.py or set an environment variable:

```
# config.py
use_web_search = True
web_search_num_results = 5
```

Then run the pipeline as usual:

```
.venv/bin/python3 -m machine_learning_engineering.agent
```

The terminal output is unchanged, but final_state.json will contain web_search_results_1 and the model proposals will be influenced by real web search results.

12. Success criteria

- duckduckgo-search is installed and declared as a dependency.

- search_web and format_results_for_prompt are implemented and tested.
- use_web_search=False keeps behavior identical to before.
- use_web_search=True populates web_search_results in AgentState.
- The pipeline completes and produces submission.csv even if DuckDuckGo fails.
- All tests pass.

13. Conclusion

Integrating DuckDuckGo web search into MLE-STAR is a contained change. It adds one utility module, two config values, one prompt update, and one call inside the initialization pipeline. Because it is opt-in, it does not affect existing behavior unless the user explicitly enables it. The result is a model retriever that can ground its proposals in recent real-world information, which is exactly what the original MLE-STAR architecture intended.